72.75: Aprendizaje Automático (Machine Learning)

Horario: Miércoles 16-19h- virtual sincrónico

Profesorado de la asignatura:
Francisco Sammartino, Ing. | frperez@itba.edu.ar
Alexa Sessarego Aurich
Carlos Bibián Nogueras, PhD | cbibian@itba.edu.ar

Plan de la sesión de hoy

- Validación cruzada avanzada
- ML diagnostics
- Data-Centered Design
- Análisis de Errores

… en la sesión anterior

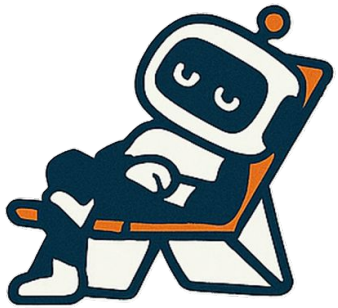
- TP2 -> clasificación supervisada

De dónde venimos

Where are we coming from

Ya **entrené**, hice **cross-validation**, elegí **modelo** e **hiperparámetros…** pero el **rendimiento** es **insuficiente**. **¿Dónde invierto ahora el esfuerzo?**

*They **do NOT build** an explicit **model** from the training data.*



Métodos avanzados de Validación Cruzada

Advanced Cross-Validation methods

Carlos Bibián Nogueras, PhD

cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

El reto tras la selección de modelo

The challenge after model selection

Dos clasificadores pueden obtener exactamente el mismo F1 y requerir intervenciones completamente diferentes.

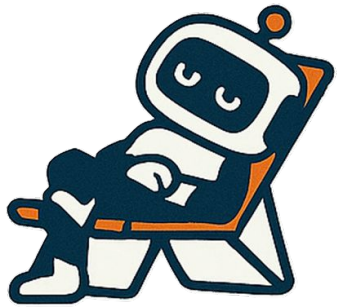
Uno puede estar limitado porque no tiene capacidad suficiente para representar la relación (X\rightarrow Y).

Otro puede aprender perfectamente training pero generalizar mal.

Otro puede discriminar muy bien positivos de negativos pero estar utilizando un threshold inadecuado.

Otro puede tener una excelente performance global pero fallar sistemáticamente en un determinado tipo de observaciones.

Por eso, la **métrica final detecta que existe un problema; no identifica cuál es el problema**.



¿Qué está limitando mi performance?

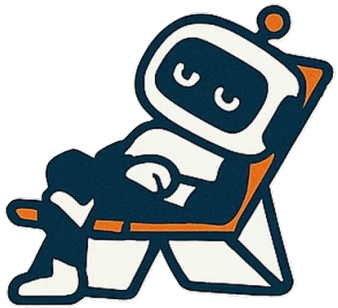
What is currently limiting performance?

El training set es sólo una muestra finita de esa distribución. Esto introduce tres fuentes conceptualmente distintas de error:

- error asociado al hecho de aprender a partir de una muestra finita (variance, cambia para otra muestra)
- error asociado al modelo o hipótesis elegida (bias)

- error inherente al problema/datos (error no explicable/reducible a partir de X)

Necesitamos un primer diagnóstico para ver qué tipo de error domina



¿Qué está limitando mi performance?

What is currently limiting performance?

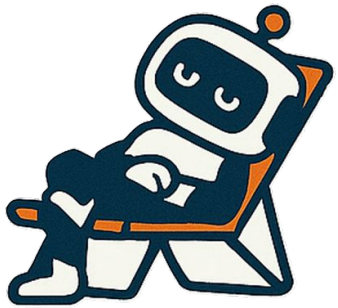
El training set es sólo una muestra finita de esa distribución. Esto introduce tres fuentes conceptualmente distintas de error:

- error asociado al hecho de aprender a partir de una muestra finita (variance, cambia para otra muestra)
- error asociado al modelo o hipótesis elegida (bias)

- error inherente al problema/datos (error no explicable/reducible a partir de X)

Necesitamos un primer diagnóstico para ver qué tipo de error domina

¿Alguna idea?



Underfitting - high bias

Are we, at least, learning well enough the training set?

No es nuestro objetivo último, pero es un buen primer paso

Si la performance en train no es buena, aquí tenemos la primera limitación

-> Apunta a una situación **de high bias (underfitting)**:

la combinación de **feats + clasificadores evaluados + optimización de hiperparámetros es incapaz extraer suficiente información de los datos**

¿Posibles causas?

Underfitting - high bias

Are we, at least, learning well enough the training set?

No es nuestro objetivo último, pero es un buen primer paso

Si la performance en train no es buena, aquí tenemos la primera limitación

-> Apunta a una situación **de high bias (underfitting)**:

la combinación de **feats + clasificadores evaluados + optimización de hiperparámetros es incapaz extraer suficiente información de los datos**

Posibles causas:

Capacidad insuficiente para extraer la información disponible en X

- modelo demasiado simple/inadecuado
- regularización excesiva/hiperparámetros demasiado restrictivos
- optimización insuficiente (NeuralNets))

Información insuficiente en X para predecir Y al nivel deseado

- features poco informativas
- demasiado ruido respecto a la señal/información (en X o en Y!!)
- límite intrínseco del problema

Overfitting - high bias

Are we generalizing to new data?

Si tenemos diferencia entre train y validation -> Generalization Gap

El modelo es capaz de extraer la información para explicar los datos de training
... pero parte de esa información que utiliza no está en los datos de validación (aprende algo de ruido)

Esta es una situación de **high variance / overfitting** (si repetimos esto para varios subconjuntos de datos, el error variará mucho)

¿Posibles causas?

Overfitting - high bias

Are we generalizing to new data?

Si tenemos diferencia entre train y validation -> Generalization Gap

El modelo es capaz de extraer la información para explicar los datos de training
... pero parte de esa información que utiliza no está en los datos de validación (aprende algo de ruido)

Esta es una situación de **high variance / overfitting** (si repetimos esto para varios subconjuntos de datos, el error variará mucho)

Posibles causas:

El modelo aprende ruido

- modelo demasiado flexible/complejo
- alta dimensionalidad ->relacion nfeats vs ndatos

Aprende información real, pero que no generaliza

- distribution shift
- leakage

Diagnóstico de modelos - Resumen

Model Diagnostics - Summary



¿Aprendemos, al menos, el training set?

Are we, at least, learning the training set?

No es nuestro objetivo último, pero es un buen primer paso

Si la performance en train no es buena, aquí tenemos la primera limitación

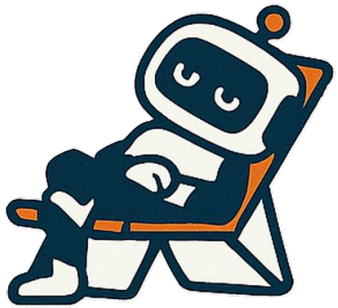
-> Apunta a una situación **de High Bias (underfitting)**:

la combinación de **feats + clasificadores evaluados + optimización de hiperparámetros es incapaz extraer suficiente información de los datos**

Posibles causas:

- modelo demasiado simple
- regularización excesiva / hiperparámetros demasiado restrictivos
- representación de entrada insuficiente / features poco informativas
- optimización incorrecta

Es decir: o falta información (X, features), o falta capacidad para extraerla



Diagnóstico de ML
ML Diagnostics

Carlos Bibián Nogueras, PhD
cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

El error de validación no es el final del proyecto
Definir baseline performance

Una buena práctica al principio del proyecto es saber cual es el **Human Level Performance (HLP)**

Esto nos sirve para decidir dónde centrar nuestros esfuerzos, que performance es más mejorable

¿cuando es útil HLP?

Unstructured data: Imágenes, Audio, Texto... en este tipo de datos, los humanos somos muy buenos, HLP es una buena forma de baseline performance

Structured data: tabla con datos de usuario o productos

aquí los humanos no somos tan buenos, así que HLP no va a ser tan útil

Human Level Performance (HLP)

Cómo calcular el HLP:

Buscar en la literatura o internet/open source

-con esto además podríamos tener una primera versión (algoritmo/modelo open source) rápidamente para tener como referencia

Si no, podemos usar una performance de una versión previa (si la tenemos)

En algunos casos, estas performance (HLP por ejemplo) nos da una idea del ***error irreducible***.

-> Nos ayuda a **decidir en qué parte del modelo o de los datos enfocarnos para reducir error**

Plantear un problema y exigir un 90% de performance no es la mejor idea:

Primero es recomendable conocer un baseline y donde está el estado del arte o el HLP, y a partir de allí definir el “objetivo”

¿Cómo continuamos?

Imaginemos que hemos recolectado unos datos, seleccionado unas características e implementado un algoritmo pero no estamos satisfechos con el desempeño: ¿Cómo continuamos?

Generalmente se suelen seguir una de estas estrategias:

-Conseguir más datos

-Probar un número menor de características

-Probar a añadir características (o variaciones de ellas)

-Cambiar hiperparámetros para que sean menos (o más) restrictivos

Pero… ¿Cuál es la mejor en nuestro caso? ¿cómo elegimos?

Diagnóstico de un modelo de ML
ML Diagnostics

Una serie de tests para entender que está (y no está) funcionando en nuestro modelo y ayudarnos a tomar decisiones sobre cómo seguir en el proyecto

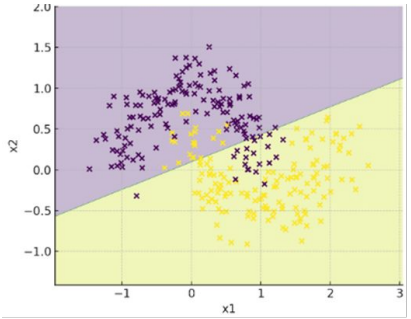
Lleva tiempo ya que es un poco más “artesanal” pero es una herramienta útil y con un impacto importante en el funcionamiento final del modelo

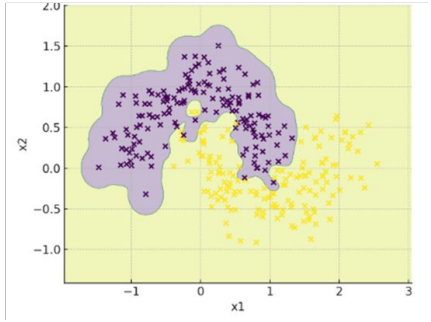
Diagnóstico de un modelo de ML
Generalización

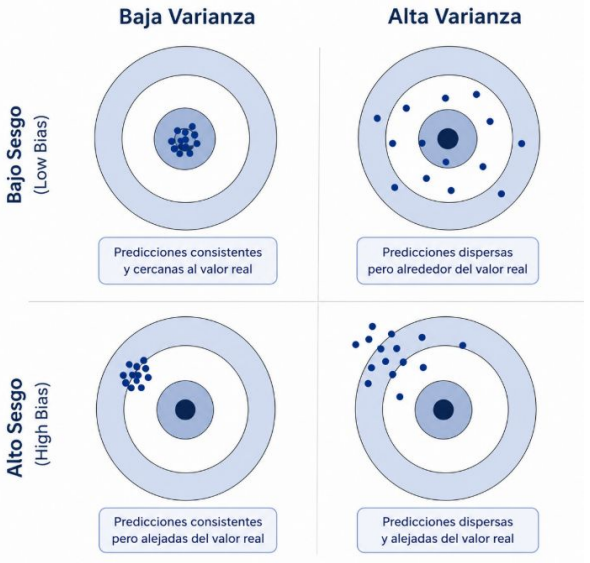
El factor crítico de nuestro modelo va a ser su capacidad para generalizar a datos nuevos

High Bias (sesgo) -> el modelo genera predicciones con un error elevado y consistente (underfitting)

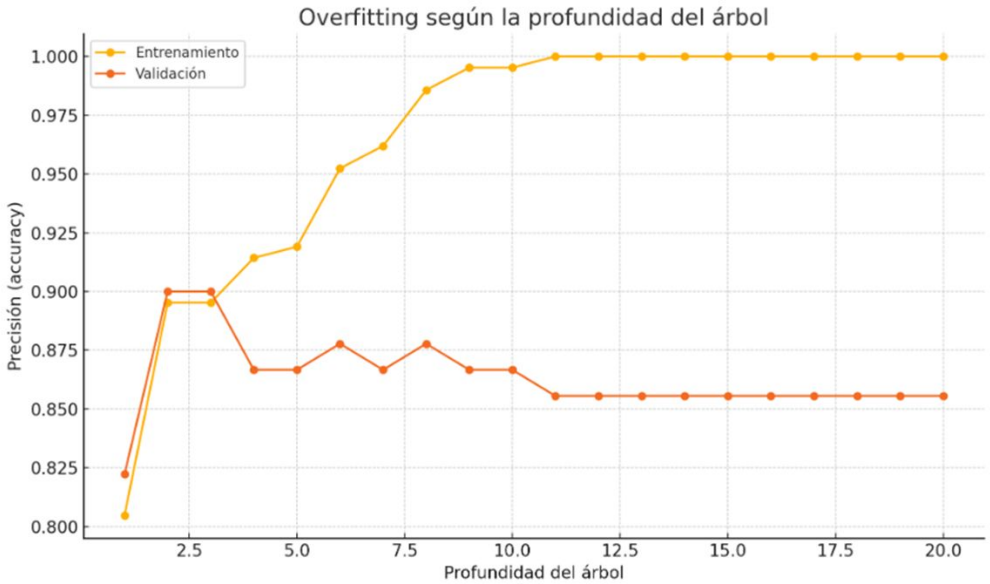
High Variance (varianza) -> el modelo es demasido sensible a los datos de entrenamiento y genera predicciones ruidosas (overfitting)







Bias vs Varianza
Generalización



La precisión en el **entrenamiento** siempre se reduce cuando se aumenta la capacidad de un algoritmo para aprender fronteras complejas

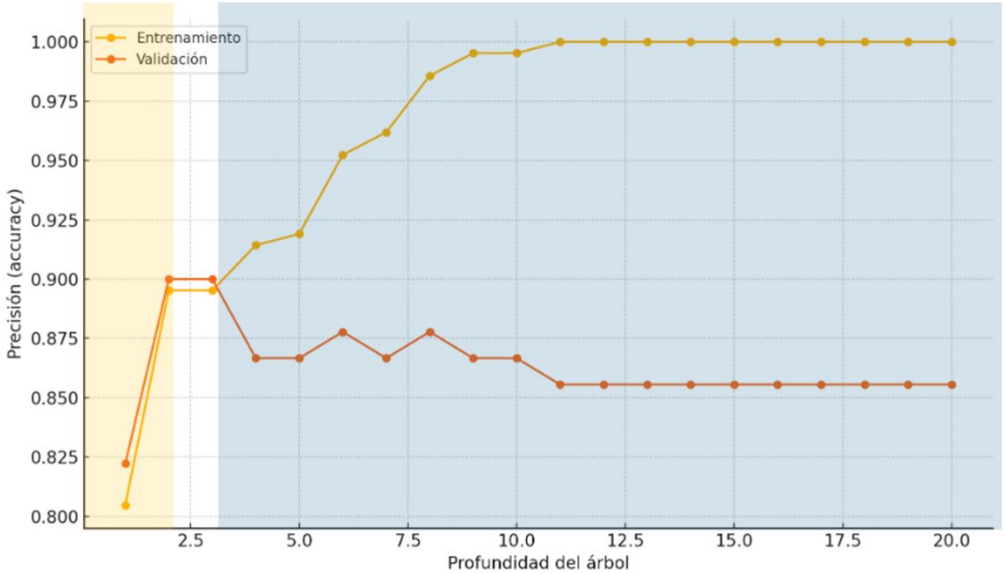
La de **validación** es distinta

->empieza a bajar cuando el modelo está sobreajustando los datos de entrenamiento

Bias vs Varianza
Generalización

Underfitting
(high bias)

Overfitting
(high variance)



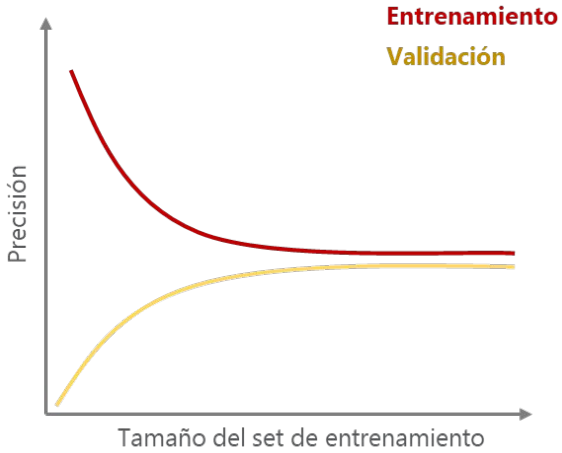
Underfitting: errores parecidos y altos en entrenamiento y validación

Overfitting: errores distintos (gap) entre entrenamiento y validación

Curvas de aprendizaje
Learning Curves

Es una gráfica útil para entender cómo cambia la performance en función del tamaño del dataset

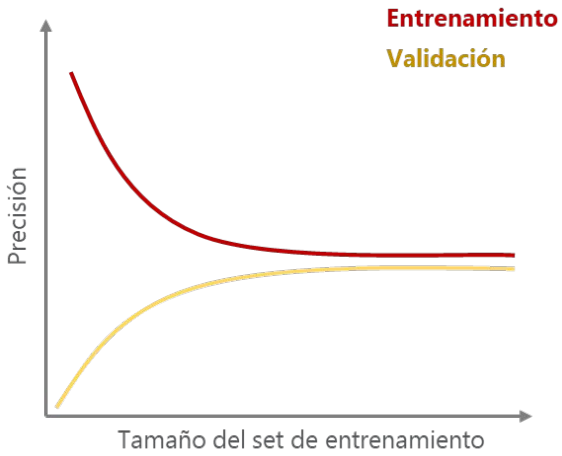
Baja precisión -> **underfitting**



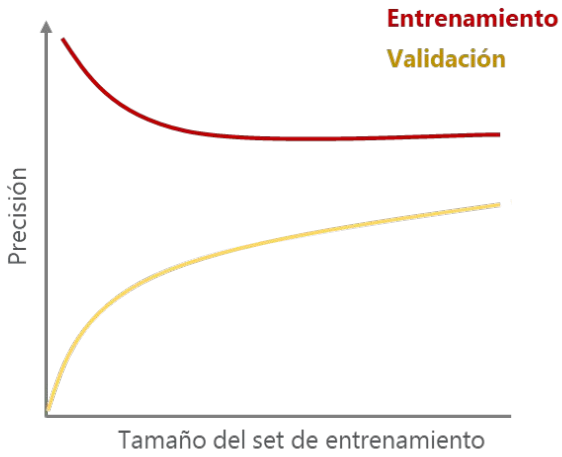
Curvas de aprendizaje
Learning Curves

Es una gráfica útil para entender cómo cambia la performance en función del tamaño del dataset

Baja precisión -> **underfitting**



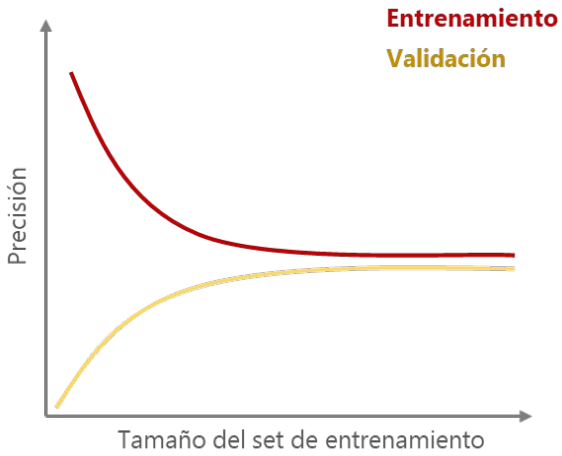
Entrenamiento-validación gap -> **overfitting**



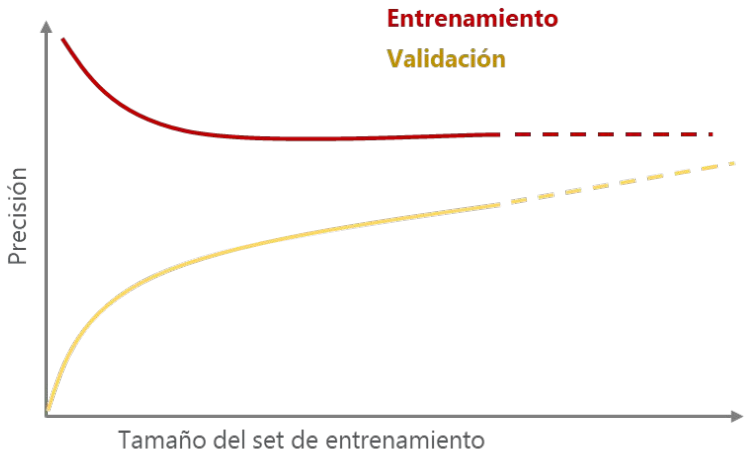
Curvas de aprendizaje
Learning Curves

Es una gráfica útil para entender cómo cambia la performance en función del tamaño del dataset

Baja precisión -> **underfitting**



Entrenamiento-validación gap -> **overfitting**



Siguientes pasos

Una vez diagnosticado si estamos en una situación de high bias (underfitting) o high variance (overfitting), podemos elegir (y descartar) algunas de las opciones que hemos visto antes:

- **Conseguir más datos de entrenamiento**
- **Usar menos features**
- **Añadir features**
- **Combinaciones de features**
- **Hiperparámetros más restrictivos**

Siguientes pasos

Una vez diagnosticado si estamos en una situación de high bias (underfitting) o high variance (overfitting), podemos elegir (y descartar) algunas de las opciones que hemos visto antes:

- **Conseguir más datos de entrenamiento** ->La curva de aprendizaje muestra que train y validación están lejos todavía (overfitting) y pero siguen cambiando al aumentar datos.
- **Usar menos features**
- **Añadir features**
- **Combinaciones de features**
- **Hiperparámetros más restrictivos**

Siguientes pasos

Una vez diagnosticado si estamos en una situación de high bias (underfitting) o high variance (overfitting), podemos elegir (y descartar) algunas de las opciones que hemos visto antes:

- **Conseguir más datos de entrenamiento** ->La curva de aprendizaje muestra que train y validación están lejos todavía (overfitting) y pero siguen cambiando al aumentar datos.
- **Usar menos features** -> Si vemos signos de overfitting, es una buena estrategia. Quitar features reduce la posibilidad del modelo de encontrar patrones que sobreajusten
- **Añadir features**
- **Combinaciones de features**
- **Hiperparámetros más restrictivos**

Siguientes pasos

Una vez diagnosticado si estamos en una situación de high bias (underfitting) o high variance (overfitting), podemos elegir (y descartar) algunas de las opciones que hemos visto antes:

- **Conseguir más datos de entrenamiento** ->La curva de aprendizaje muestra que train y validación están lejos todavía (overfitting) y pero siguen cambiando al aumentar datos.
- **Usar menos features** -> Si vemos signos de overfitting, es una buena estrategia. Quitar features reduce la posibilidad del modelo de encontrar patrones que sobreajusten
- **Añadir features** ->Si hay underfitting, añadir features es una buena opción de dar más opciones al modelo de aprender
- **Combinaciones de features**
- **Hiperparámetros más restrictivos**

Siguientes pasos

Una vez diagnosticado si estamos en una situación de high bias (underfitting) o high variance (overfitting), podemos elegir (y descartar) algunas de las opciones que hemos visto antes:

- **Conseguir más datos de entrenamiento** ->La curva de aprendizaje muestra que train y validación están lejos todavía (overfitting) y pero siguen cambiando al aumentar datos.
- **Usar menos features** -> Si vemos signos de overfitting, es una buena estrategia. Quitar features reduce la posibilidad del modelo de encontrar patrones que sobreajusten
- **Añadir features** ->Si hay underfitting, añadir features es una buena opción de dar más opciones al modelo de aprender
- **Combinaciones de features** ->Si hay underfitting, combinar features es una buena opción de dar más opciones al modelo de aprender
- **Hiperparámetros más restrictivos**

Siguientes pasos

Una vez diagnosticado si estamos en una situación de high bias (underfitting) o high variance (overfitting), podemos elegir (y descartar) algunas de las opciones que hemos visto antes:

- **Conseguir más datos de entrenamiento** ->La curva de aprendizaje muestra que train y validación están lejos todavía (overfitting) y pero siguen cambiando al aumentar datos.
- **Usar menos features** -> Si vemos signos de overfitting, es una buena estrategia. Quitar features reduce la posibilidad del modelo de encontrar patrones que sobreajusten
- **Añadir features** ->Si hay underfitting, añadir features es una buena opción de dar más opciones al modelo de aprender
- **Combinaciones de features** ->Si hay underfitting, combinar features es una buena opción de dar más opciones al modelo de aprender
- **Hiperparámetros más restrictivos** -> Si hay overfitting, unos hiperparámetros más estrictos pueden

Desarrollo de ML centrado en los datos
Data-centric ML development

Carlos Bibián Nogueras, PhD
cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

Diseño centrado en Modelo vs Datos
Model vs Data-centric design

Model-centric:

- dados unos datos, encontrar el modelo (y set de hiperparametros) que mejor funcionan.
- mantener los datos fijos e iterativamente mejorar el modelo

Data-centric:

- implementamos un modelo que sabemos que funciona razonablemente bien y lo dejamos fijo
- Aplicamos distintos métodos para aumentar la calidad (o cantidad) de los datos (***añadir feats tb!***)
- Esto repercutirá en mejor performance del modelo.

Análisis de Errores
Error Analysis

El objetivo es entender qué tipo de errores comete el modelo y qué tienen en común

Sería ir un paso (o varios) más allá de la matriz de confusión

Ya que si encontramos un patrón, podremos intentar mejorar el funcionamiento del modelo en ese aspecto concreto.

Se puede hacer manualmente, viendo los errores (predicciones incorrectas) y viendo cómo son los datos y anotando tags (aspectos que pueden ocurrir)

Objetivo: ver cómo son los errores cometidos e identificar algún patrón o motivo por el que no están siendo bien clasificados

Primer paso: elegir qué factores (tags) se van a analizar

Análisis de Errores
Elección de tags (factores)

Inspección Visual:

Clase específica: hay más errores en una clase que en otra - matriz de confusión

Propiedades de la imagen: borrosa, oscura, reflejos…

Metadatos: modelo de cámara/celular, empresa, tipo de usuario…

Errores en un tipo concreto de datos: recomendaciones a jóvenes, u otro tipo de x

Análisis de Errores
Elección de tags (factores)

Inspección Visual:

Clase específica: hay más errores en una clase que en otra - matriz de confusión

Propiedades de la imagen: borrosa, oscura, reflejos…

Metadatos: modelo de cámara/celular, empresa, tipo de usuario…

Errores en un tipo concreto de datos: recomendaciones a jóvenes, u otro tipo de x

Una vez elegidos los tags

¿qué porcentaje de errores tienen este tag? - esto nos dice cuánto mejoramos la perf si solucionásemos ese error

Es decir, de todos los datos de ese tag, ¿cuantos están mal clasificados? si esto es alto, indica que no está funcionando muy bien (se puede comparar con los que no tengan ese tag)

– con este análisis podemos ver que tipo de datos priorizar al mejorar nuestro modelo/datos.

Análisis de errores
Impacto de cada Tag

¿Cómo elegir en qué Tag trabajar?

Tipo de imagen	Accuracy
Imagen limpia	96%
Bajo contraste	88%
Artefacto de movimiento	82%
Dispositivo distinto	85%
Imagen recortada/parcial	70%

Análisis de errores
Impacto de cada Tag

¿Cómo elegir en qué Tag trabajar?

Tipo de imagen	Accuracy	Human level performance
Imagen limpia	96%	97%
Bajo contraste	88%	94%
Artefacto de movimiento	82%	90%
Dispositivo distinto	85%	92%
Imagen recortada/parcial	70%	80%

Análisis de errores
Impacto de cada Tag

¿Cómo elegir en qué Tag trabajar?

Tipo de imagen	Accuracy	Human level performance	Gap to HLP
Imagen limpia	96%	97%	1%
Bajo contraste	88%	94%	6%
Artefacto de movimiento	82%	90%	8%
Dispositivo distinto	85%	92%	7%
Imagen recortada/parcial	70%	80%	10%

Análisis de errores
Impacto de cada Tag

¿Cómo elegir en qué Tag trabajar?

Tipo de imagen	Accuracy	Human level performance	Gap to HLP	% de datos	Contribución al gap
Imagen limpia	96%	97%	1%	50%	0.5%
Bajo contraste	88%	94%	6%	20%	1.2%
Artefacto de movimiento	82%	90%	8%	15%	1.2%
Dispositivo distinto	85%	92%	7%	10%	0.7%
Imagen recortada/parcial	70%	80%	10%	5%	0.5%

Análisis de errores
Cómo decidir el siguiente paso

¿Cómo elegir en qué trabajar?

Cuánto margen de mejora hay

Cual es la frecuencia de aparición de esa categoría

Cómo de fácil es mejorar la accuracy en esa categoría (quizás añadir una feature específica ayude)

Cómo de importante es mejorar esa categoría (si es muy importante podemos incluso añadir una métrica específica para eso en la selección de modelo)

Análisis de errores
Cómo decidir el siguiente paso

¿Cómo elegir en qué trabajar?

Cuánto margen de mejora hay

Cual es la frecuencia de aparición de esa categoría

Cómo de fácil es mejorar la accuracy en esa categoría (quizás añadir una feature específica ayude)

Cómo de importante es mejorar esa categoría (si es muy importante podemos incluso añadir una métrica específica para eso en la selección de modelo)

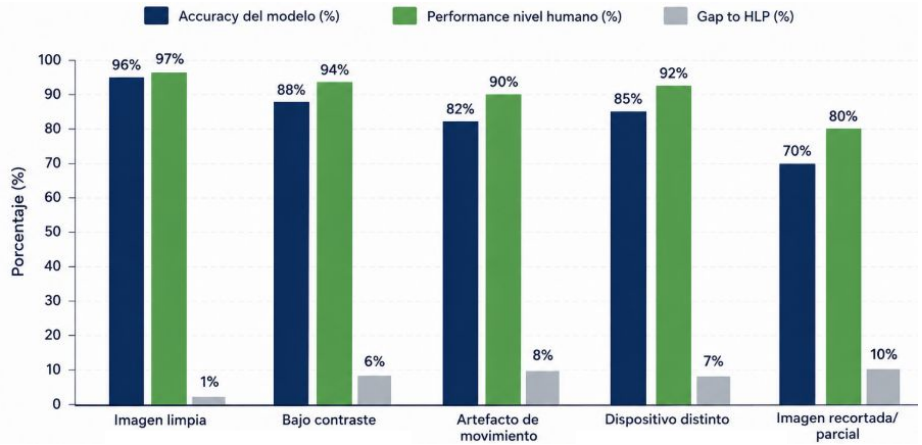
Cuando elegimos una categoría para “mejorar” la performance en ella tenemos varias opciones:

- Mejorar la calidad de las etiquetas/datos de esa categoría
- Grabar más datos de ese tipo
- Usar data augmentation

Data-centric approach
Ejemplo de análisis de errores previo

Podemos ver cómo de efectivo el modelo es en datos de cada tipo de ruido y también ver el HLP para cada tipo de ruido

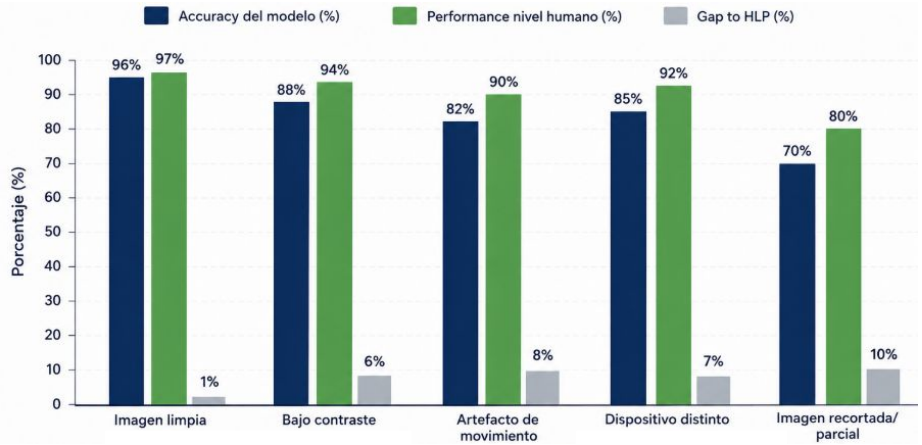
En este análisis vemos que tipos de datos representan una oportunidad de mejora más grande!



Data-centric approach
Ejemplo de análisis de errores previo

Podemos ver cómo de efectivo el modelo es en datos de cada tipo de ruido y también ver el HLP para cada tipo de ruido

En este análisis vemos que tipos de datos representan una oportunidad de mejora más grande!



Si conseguimos más datos de distintos dispositivos, mejorará el error.

A veces también hay relaciones entre “tags vecinos” que hacen que al mejorar uno mejore otro

- Quizás artefacto movimiento e imagen desenfocada o imagen recortada

Bucle de iteración de diseño
Añadir Datos

En el **diseño centrado en el modelo**:

1- Entreno modelo

2- Evalúo

3- Aplico cambios

4- Vuelvo al 1...

Bucle de iteración de diseño
Añadir Datos

En el **diseño centrado en el modelo**:

1- Entreno modelo

2- Evalúo

3- Aplico cambios

4- Vuelvo al 1...

En el **diseño centrado en los Datos**:

1- Entreno modelo

2- Evalúo - Error Análisis

3- Añado/Mejoro/Modifico los datos

4- Vuelvo al 1...

Para muchas aplicaciones, este data iteration loop es más efectivo y mejora más rápido que el model iteration approach (siempre con un hiperparameters search y clasificador robusto!!)

Bucle de iteración de diseño
Añadir Datos

En el **diseño centrado en el modelo**:

1- Entreno modelo

2- Evalúo

3- Aplico cambios

4- Vuelvo al 1...

En el **diseño centrado en los Datos**:

1- Entreno modelo

2- Evalúo - Error Análisis

3- Añado/Mejoro/Modifico los datos

4- Vuelvo al 1...

Para muchas aplicaciones, este data iteration loop es más efectivo y mejora más rápido que el model iteration approach (siempre con un hiperparameters search y clasificador robusto!!)

El análisis de errores es más difícil si no tenemos acceso a calcular HLP

Podemos mirar papers o problemas similares para encontrar features útiles para nosotros!

-> Añadir features tambien es añadir datos!

Bucle de iteración de diseño
Añadir Datos

En el diseño centrado en el modelo:

1- Entreno modelo

2- Evalúo

3- Aplico cambios

4

En el diseño centrado en los Datos:

1- Entreno modelo

2- Evalúo - Error Análisis

3- Añado/Mejoro/Modifico los datos

Sin embargo, no siempre es posible conseguir nuevos datos...

-> **Data Augmentation**

El análisis de errores es más difícil si no tenemos acceso a calcular HLP

Podemos mirar papers o problemas similares para encontrar features útiles para nosotros!

-> Añadir features tambien es añadir datos!

Aumentación de Datos
Data Augmentation

Carlos Bibián Nogueras, PhD
cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

Aumentación de datos
Definición

Herramienta muy efectiva para conseguir más datos, especialmente en unstructured data (no tabular)

Consiste en crear datos sintéticos juntando señal y ruido (obtenidos por separado)

Es recomendable comprobar primero:

Si el dato sintético es realista (imagen, audio,,,)

Si un humano puede hacer esa tarea? x->y mapping existe

También, en el análisis de errores, si funciona el algoritmo mal ahora en ese tipo de dato?, sino no hace falta

En campos como medicina, problemas industriales y datasets pequeños/medianos, suele ser extremadamente útil!

Data Augmentation
Ejemplos de uso según los datos

Audio:

a partir de un audio, generamos varios audios distintos añadiendo distintos niveles o tipos de ruido, recortar un segmento (windowing)

Data Augmentation
Ejemplos de uso según los datos

Audio:

a partir de un audio, generamos varios audios distintos añadiendo distintos niveles o tipos de ruido, recortar un segmento (windowing)

Imagen:

rotar, cambiar brillo, enfoque (blur) o recortar un trozo (crop), añadir ruido

Data Augmentation
Ejemplos de uso según los datos

Audio:

a partir de un audio, generamos varios audios distintos añadiendo distintos niveles o tipos de ruido, recortar un segmento (windowing)

Imagen:

rotar, cambiar brillo, enfoque (blur) o recortar un trozo (crop), añadir ruido

NLP:

reemplazar palabras por sinónimos, borrar algunas palabras aleatoriamente, paragraasear el texto usando LLM….

Data Augmentation
Restricciones

La idea clave es que los nuevos datos deben:

- preservar la etiqueta (y)
- respetar la fisiología/física del problema
- no cambiar la distribución esperada del problema

Importante, como en los ejemplos vistos anteriormente, que no cambien el significado físico/semántico de los datos

Por eso se aplica principalmente a nivel de datos (unstructured data), una vez que tenemos una “tabla de números” es mucho más complicado aplicarlo, ya que se pueden generar relaciones extrañas entre variables/datos

Data Augmentation
Riesgos

Garbage in, garbage out

Ocurre cuando aumentación no simula datos reales y confunde al algoritmo

por ejemplo, rotar dígitos 180 puede hacer que confundamos un 6 y un 9

Data Augmentation
Riesgos

Garbage in, garbage out

Ocurre cuando aumentación no simula datos reales y confunde al algoritmo

por ejemplo, rotar dígitos 180 puede hacer que confundamos un 6 y un 9

Leakage

1) Utilizamos los datos de un paciente para generar más datos

2) mezclamos entre train y validation datos del mismo paciente…

Corremos el riesgo de que el algoritmo “aprenda” el paciente

(similar al caso de group CV o LOGO)

Data Augmentation
Riesgos

Garbage in, garbage out

Ocurre cuando aumentación no simula datos reales y confunde al algoritmo

por ejemplo, rotar dígitos 180 puede hacer que confundamos un 6 y un 9

Leakage

1) Utilizamos los datos de un paciente para generar más datos

2) mezclamos entre train y validation datos del mismo paciente…

Corremos el riesgo de que el algoritmo “aprenda” el paciente

(similar al caso de group CV o LOGO)

Cambio de distribución

Si se aplica en exceso (en cantidad o tamaño de las perturbaciones)

->puede cambiar la distribución de los datos, y alejarse del escenario real del problema

Data Augmentation
Datos Tabulares

Por eso se aplica principalmente a nivel de datos, una vez que tenemos una “tabla de números” es mucho mas complicado aplicarlo, ya que se pueden generar relaciones extrañas entre variables/datos

En estos casos, es más interesante el **feature engineering**

Para datos tabulares, hay métodos como **SMOTE o GANs**, que generas datos interpolando vecinos

Funcionan bien, pero **debemos tener en cuenta que pueden generar estas datos imposibles** y hacer que el modelo generalice peor

Por ejemplo, en EEG:

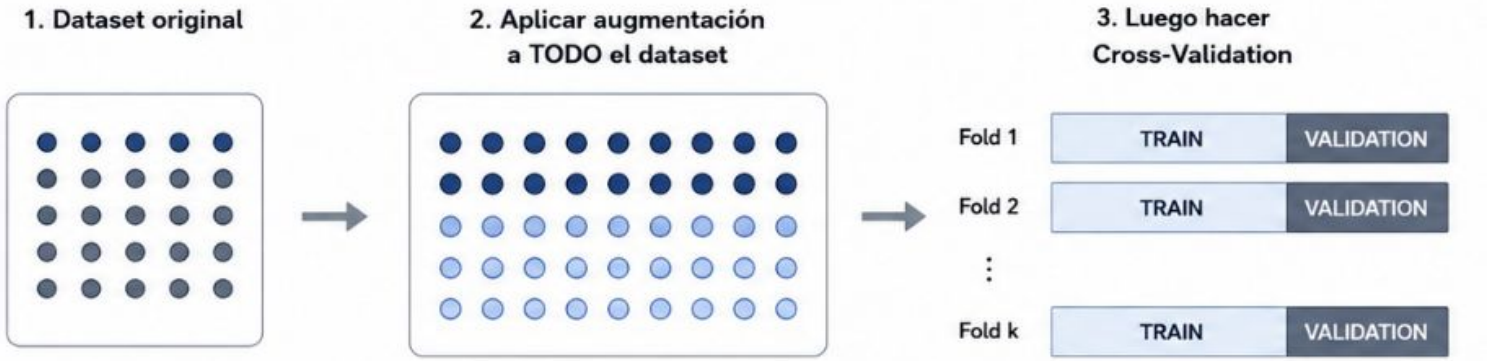
- si hacemos augmentation después de calcular potencia de cada banda de frecuencia, perdemos coherencia fisiológica de los datos
- es mucho más aconsejable hacer augmentation en la señal cruda

Data Augmentation
¿Afecta al Split de alguna forma?

¿Como haríamos el data augmentation, en que momento de nuestro pipeline?

Data Augmentation
¿Afecta al Split de alguna forma?

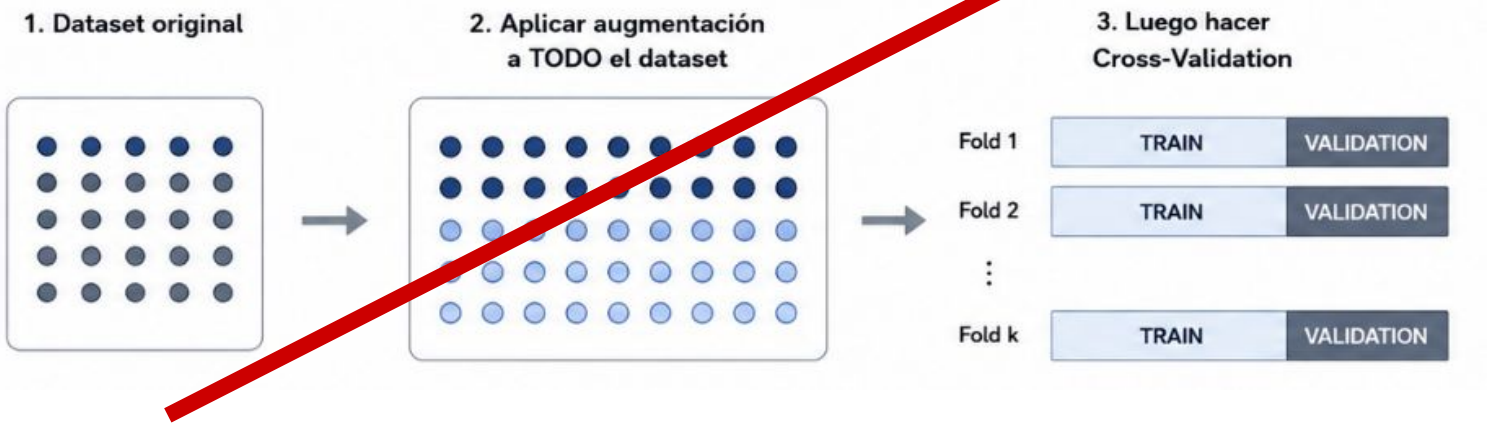
¿Como haríamos el data augmentation, en que momento de nuestro pipeline?



Data Augmentation
¿Afecta al Split de alguna forma?

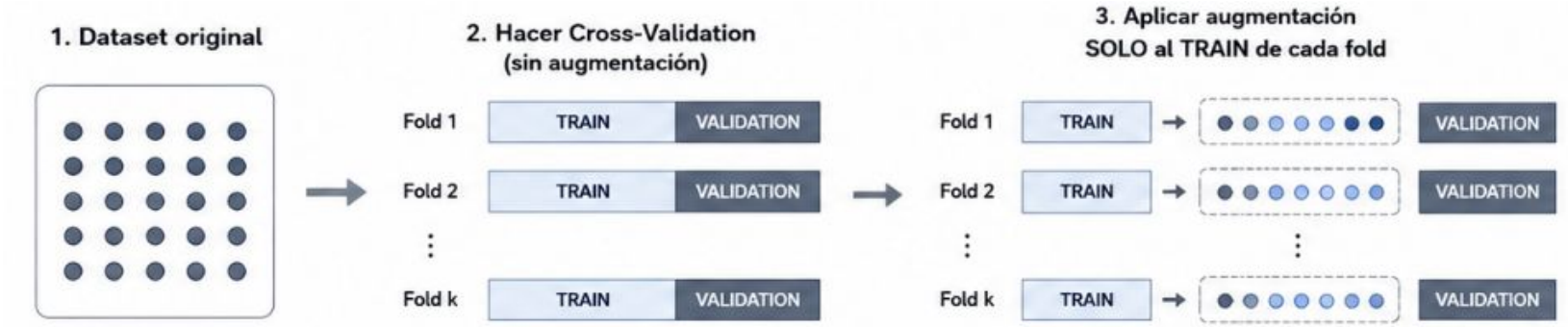
¿Como haríamos el data augmentation, en que momento de nuestro pipeline?

¡Así no! se produciría **leakage** entre train y validación **si hay datos sintéticos en validación** creados a partir de un dato real que está en train (o datos sintéticos "hijos" del mismo dato en train y validación)



Data Augmentation
Implementación correcta

Es importante que la validación (y test) no contengan datos aumentados, ya que no son reales y pueden sesgar la estimación de la capacidad de generalización



Data Augmentation
Implementación correcta

Es importante utilizar data augmentation **sólo en datos de train y dejar validación y test con datos "reales"**

Si no lo hacemos, podemos obtener una estimación sesgada de cómo de bien generaliza el modelo.

Esto ocurre con frecuencia en algunos papers…

Data Augmentation
Test-time augmentation (TTA)

Un caso especial de esto es utilizar las aumentaciones (modificaciones) de la imagen o dato original para hacer varias predicciones y luego promediarlas/combinarlas

En lugar de predecir sobre un dato, predices sobre ese dato y sus variaciones y luego combinas esas predicciones

Data Augmentation
Cuando utilizarlo

Es especialmente útil cuando:

- Hay pocos datos
- Hay riesgo de overfitting
- Recolectar datos es caro (ej, Neurociencia, medicina…)

Es menos útil cuando el dataset es grande o los datos son tabulares y/o con features abstractas

Data Augmentation
Conclusiones

La data augmentation puede **mejorar generalización y reducir overfitting**.

Debe aplicarse **sólo sobre el conjunto de entrenamiento**.

La **validación y el test** deben realizarse sobre datos reales **no aumentados**.

Las transformaciones deben **respetar la semántica y la fisiología del problema**.

Una augmentation incorrecta puede introducir *data leakage* y métricas artificialmente infladas.

Muchos vs Buenos Datos
From Big Data to Good Data

Big Data siempre ayuda (si tenemos los recursos para procesarlo y un algoritmo para ello), pero no siempre es posible

Tener buenos datos, nos ayuda siempre y es más sencillo de conseguir!

-> Un algoritmo correcto, con buenos datos va a funcionar mucho mejor que un muy buen algoritmo con datos no tan buenos. -

> No obsesionarse con el modelo al principio, porque poder empezar a iterar en el loop nos va a permitir mejorar mas rápido y entender mejor nuestros datos

¿Cómo podemos definir “buenos datos”?

La relación x->y es consistente (HLP>random)

Cubre casos importantes de uso (tipos de clientes etc)

Tiene un tamaño razonable

Relación train-validation y train-test y train-deployment se mantiene

(data drift y concept drift, siguiente clase)

Métodos avanzados de Validación Cruzada

Advanced Cross-Validation methods

Carlos Bibián Nogueras, PhD

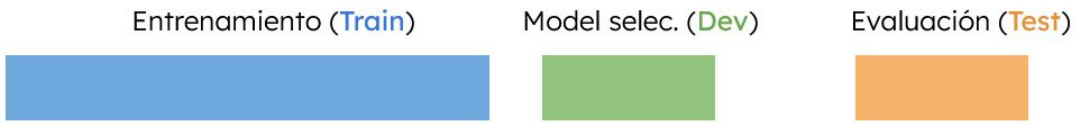
cbibian@itba.edu.ar |

Instituto Tecnológico de Buenos Aires

¿Porque necesitamos Validación?
Resumen

Medir/testear la capacidad de generalización

¿Cómo se va a comportar mi modelo en datos nuevos?





¿Cómo utilizamos la Validación?
Resumen Cross Validation

Cross validation:



Evaluamos (y elegimos) modelos, no fronteras de decisión.

Ej: un RF con estas features generaliza lo suficientemente bien? esa es la pregunta que responde validación.

La **validación no es para definir la frontera de decisión**, sino el modelo (feats+algoritmo) que usaré para encontrar esa frontera

Una vez respondida esa pregunta, entremanos el algoritmo elegido con esas features en TODOS los datos (menos los de test), y ese será mi modelo final para deploy

¿Como debe ser la Validación?
Resumen

Train - Validation y Test deben ser iguales:

Deben mostrar los mismos datos, para que lo que se aprenda en Train pueda ser aplicado en Validación y Test

Hasta ahora, enfoque más simple,

- separar los datos aleatoriamente
- hacerlo de forma sucesiva para que todos los datos se usen una vez en Validación (Cross-Validation)

¿Como debe ser la Validación?
Resumen

Train - Validation y Test deben ser iguales:

Deben mostrar los mismos datos, para que lo que se aprenda en Train pueda ser aplicado en Validación y Test

Hasta ahora, enfoque más simple,

- separar los datos aleatoriamente
- hacerlo de forma sucesiva para que todos los datos se usen una vez en Validación (Cross-Validation)

Más avanzado: stratified Cross-Validation -> mantiene la proporción de clases en train-valdation-test

¿Como debe ser la Validación?
Resumen

Train - Validation y Test deben ser iguales:

Deben mostrar los mismos datos, para que lo que se aprenda en Train pueda ser aplicado en Validación y Test

En otras palabras:

La relación Train-Validation debe ser la misma que habrá entre Train-Test

Elegir el tipo Split

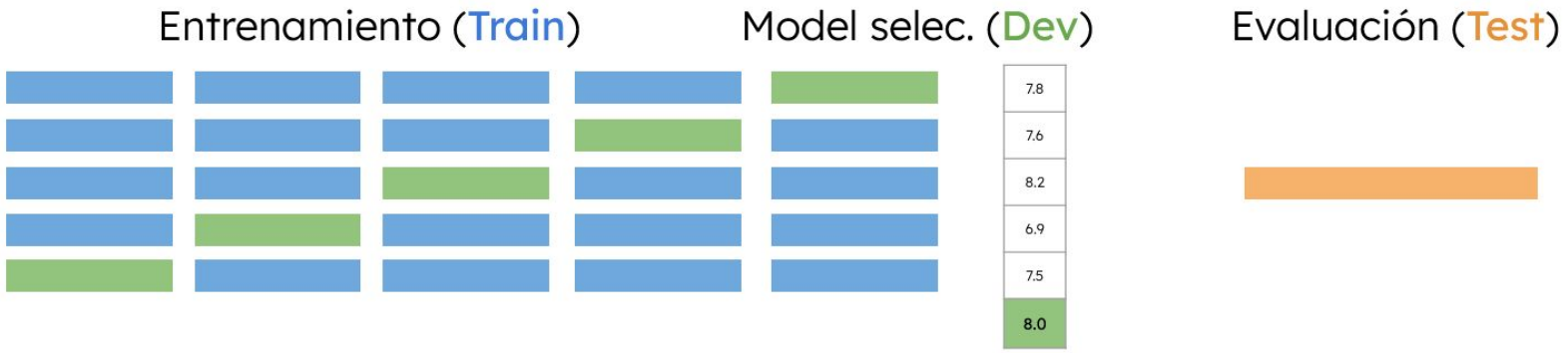
La relación Train-Validation debe ser la misma que habrá entre Train-Test

La selección del Split define el uso que le vamos a dar al modelo

ó

El uso que le vayamos a dar al modelo, define el Split que debemos usar

k-fold Validación Cruzada aleatoria
Random k-fold Cross-Validation



k-fold Validación Cruzada Estratificada
Stratified k-fold Cross-Validation

Problema del random split

La proporción de clases puede según la seed, especialmente en datasets pequeños o desbalanceados.

Ejemplo:

Fold A → 5% positivos

Fold B → 20% positivos

Stratified-CV -> mantiene la proporción de clases en train-validation-test

Es recomendable usarlo siempre, especialmente en datasets pequeños/medianos (donde el riesgo de que al hacerlo aleatoriamente se desbalanceen es más grande)

Recomendable en datasets pequeños o desbalanceados (medicina, fraude, diagnosis….)

Grupos/Cohortes
Groups/Cohorts

En ocasiones, las muestras no son independientes

Pueden existir grupos/cohortes/estructuras compartidas que introduzcan relaciones entre las muestras

Por ejemplo:

Hospitales

Escuelas

Países

Dispositivos

La relación Train-Validation debe ser la misma que habrá entre Train-Test

Grupos/Cohortes
Groups/Cohorts

Estos grupos comparten sesgos, correlaciones, cambios de distribución....

si no se tiene en cuenta, se puede producir leakage indirecto:

El modelo aprende el hospital o el scanner o el país, en lugar de aprender el problema real

¿Es esto un problema?

Grupos/Cohortes
Groups/Cohorts

Estos grupos comparten sesgos, correlaciones, cambios de distribución....

si no se tiene en cuenta, se puede producir leakage indirecto:

El modelo aprende el hospital o el scanner o el país, en lugar de aprender el problema real

¿Es esto un problema?

Si, si no se mantiene la relación Train-Validation en Train-Test/Deployment

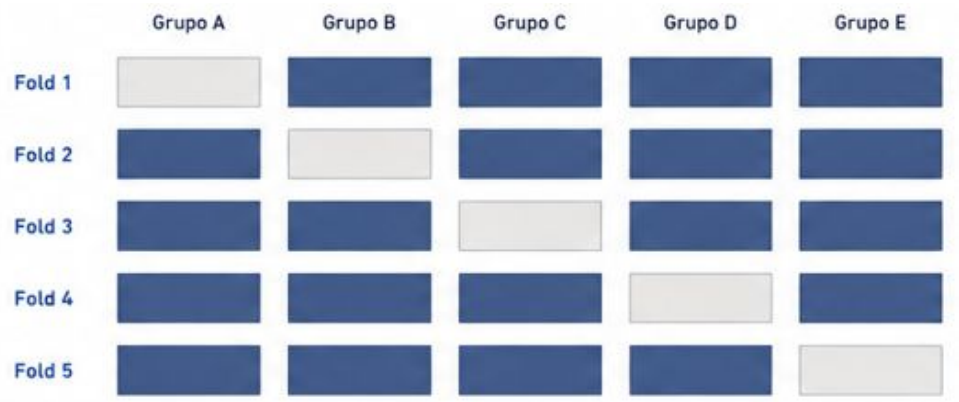
Es un problema ya que el sistema puede usar información que luego no tendrá en su aplicación final!

Validación Cruzada por Grupos
Group Cross-Validation

Métodos como:

-Group K-Fold (cada fold es un grupo)

-Leave-One-Group-Out (LOGO) - El conjunto de validación serán los datos de un grupo en cada iteración del CV



Grupos/Cohortes
Ejemplo

Tenemos información de 5 escuelas distintas en nuestro dataset

El sistema queremos utilizarlo en otras escuelas, no incluidas en el dataset:

Debemos mantener en Train-Validation esa relación entre train-deployment, por lo que debemos usar un Group K-fold Cross-Validation

Sino, corremos el riesgo de que aprenda a funcionar en esos 5 hospitales pero no en otros, debemos validar también esa capacidad de aprender de unos hospitales y generalizar a otros, que es el uso final que va a tener el sistema

Cohortes Temporales
Temporal Cohorts

Si el factor tiempo es relevante, debemos saber si nuestro sistema es capaz de generalizar a datos nuevos “del futuro”

¿Cómo hacemos eso?

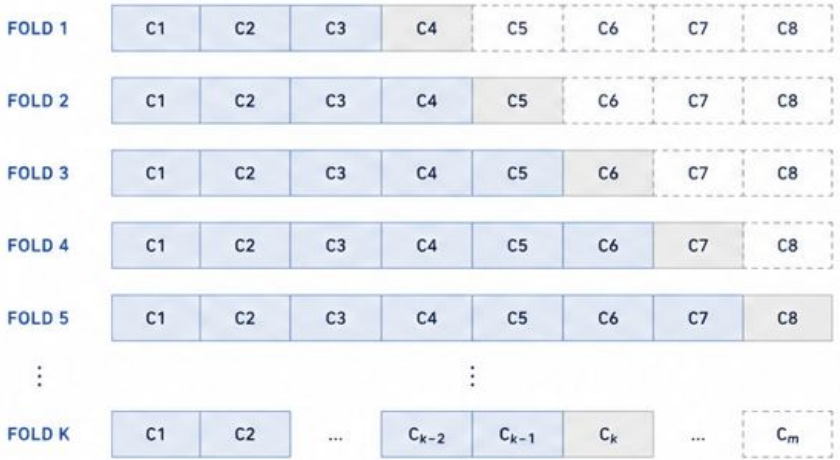
Cohortes Temporales
Temporal Cohorts

Si el factor tiempo es relevante, debemos saber si nuestro sistema es capaz de generalizar a datos nuevos “del futuro”

¿Cómo hacemos eso? La Validación contiene siempre datos posteriores al Train, siempre!

Time-Series Cross-Validation

Walk-Forward Validation



¿Cómo elegir Split?

Por ejemplo, ¿Cómo elegimos el Split cuando hay varios niveles de Cohortes?

Siempre responder la pregunta: **¿A qué queremos que generalice el sistema cuando se use?**

¿Cómo elegir Split?

Por ejemplo, ¿Cómo elegimos el Split cuando hay varios niveles de Cohortes?

Siempre responder la pregunta: ¿A qué queremos que generalice el sistema cuando se use?

¡En ocasiones, elegir el Split correcto tiene un impacto mayor a elegir uno u otro modelo!

No existe un Split universalmente correcto:

depende del uso que se le vaya a dar al sistema/Deployment